



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**EduVerse: AI-Powered Adaptive Learning and Student Success
Analytics Platform**

**Agile Sprint Planning & User Story Workshop
(CO1, ASSESSMENT TOOL 2)**

Submitted in the partial fulfilment for the Course of
CSA1610 - SOFTWARE ENGINEERING

to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
CSE(Ai)

Submitted by
H.Ganesh Kumar (192572027)

Under the Supervision of
Dr. K Anita Davamani

INDEX

- 1) Identify the Project Vision and Stakeholders
 - 1.1 Project Vision
 - 1.2 Key Stakeholders
- 2) Create Epics and User Stories
- 3) Define Acceptance Criteria
- 4) Prioritize the Product Backlog
- 5) Plan the Sprint Backlog
- 6) Estimate Effort Using Story Points
- 7) Present the Sprint Goal and Expected Deliverables
 - 7.1 Sprint 1 Goal
 - 7.2 Expected Deliverables (End of Sprint)
 - 7.3 Definition of Done (DoD) for Sprint 1

1. Identify the Project Vision and Stakeholders

1.1 Project Vision

The vision for EduVerse is to transform education delivery from a rigid, one-size-fits-all model into a proactive, personalized, and data-driven learning ecosystem. By leveraging Artificial Intelligence, Machine Learning, and real-time learning analytics, the platform continuously adapts content and pacing to each student's mastery level, detects early signs of academic risk, and automates the administrative overhead of tracking progress. The ultimate objective is to maximize student success and retention, empower educators with actionable insights, and ensure strict compliance with data privacy regulations (FERPA/GDPR) through transparent, auditable data handling.

Key Objectives:

- Deliver personalized learning paths that adapt in real time to each student's demonstrated mastery level.
- Preemptively identify at-risk students using predictive analytics on engagement and performance data.
- Relieve instructors of manual grading and progress-reporting overhead through AI-assisted automation.
- Ensure a scalable, secure, cloud-native architecture that safeguards student data and meets education data-privacy regulations.

1.2 Key Stakeholders

Academic Deans / Institution Leadership (High Influence, High Interest):

Primary sponsor and ultimate decision-maker. Concerned with overall academic outcomes, budget adherence, institutional reputation, and regulatory compliance.

Instructors / Teachers (Low Influence, High Interest):

The primary end-users of the analytics dashboards and grading tools. Their main concern is ease of use, time saved on administrative tasks, and the reliability of AI-generated insights.

IT Administrators & Data Security Team (High Influence, Medium Interest):

Responsible for maintaining the cloud infrastructure. Concerned with system uptime, scalability, data encryption, and access control.

Students / Learners (Medium Influence, High Interest):

Primary consumers of the adaptive content and feedback. Focused on clear progress visibility, fair assessment, and timely, useful feedback.

Parents / Guardians (Medium Influence, Medium Interest):

Recipients of progress reports and alerts. Focused on staying informed about their child's academic standing without needing to interpret raw data.

Academic Data Analysts / Compliance Officers (Medium Influence, High Interest):

Responsible for validating model accuracy and ensuring all student data collection and processing complies with FERPA/GDPR and institutional policy.

2. Create Epics and User Stories

The platform's requirements have been decomposed into major Epics, each further broken down into actionable User Stories following the standard Agile format.

Epic 1: Adaptive Learning Path Engine

- **As a Student**, I want the system to recommend my next learning module based on my current mastery level, so that I can focus on topics I have not yet mastered instead of repeating material I already know.
- **As a Content Designer**, I want to tag learning content with difficulty level and prerequisite metadata, so that the adaptive engine can sequence modules in the correct learning order.
- **As a Instructor**, I want to manually override the AI-recommended learning path for a specific student, so that I can account for context the system does not have visibility into.

Epic 2: Student Success Prediction & Early-Warning System

- **As a Instructor**, I want to see an automatically updated risk score for each student every week, so that I can identify and support students before they fall behind or fail.
- **As a Academic Advisor**, I want to receive an automated alert when a student's engagement drops sharply, so that I can schedule a timely check-in meeting.
- **As a Dean**, I want a cohort-level dashboard showing trends in at-risk students across departments, so that I can allocate tutoring and support resources strategically.

Epic 3: AI-Powered Assessment & Feedback

- **As a Student**, I want to receive instant, personalized feedback on my quiz answers, so that I can understand and correct my mistakes immediately.
- **As a Instructor**, I want short-answer responses to be auto-graded using NLP, so that I save significant grading time on large classes.
- **As a Content Designer**, I want the system to auto-generate additional practice questions from a given topic, so that students get more targeted practice material without manual authoring.

Epic 4: Engagement Analytics & Behavioral Tracking

- **As a Learning Analytics Officer**, I want to track time-on-task and click-stream data for each student, so that I can model engagement patterns that feed the prediction engine.
- **As a Instructor**, I want a heatmap showing which lesson sections students re-watch or skip, so that I can identify and improve weak points in the course content.
- **As a Student**, I want to see my own progress, mastery levels, and study streaks, so that I stay motivated and aware of my own learning journey.

Epic 5: Administrative Automation & Reporting Dashboards

- **As a Instructor**, I want to auto-generate an end-of-term progress report for each student, so that I do not need to manually compile grades and comments.

- **As a Parent**, I want to receive a weekly digest summarizing my child's progress, so that I stay informed without needing to log into the platform daily.
- **As a Compliance Officer**, I want every access to student data to be logged with a timestamp and user ID, so that I can demonstrate FERPA/GDPR compliance during an audit.

3. Define Acceptance Criteria

Below are clear, measurable acceptance criteria using the Given–When–Then framework for selected high-priority user stories from the backlog.

User Story: As a Student, I want the system to recommend my next learning module based on my current mastery level, so that I can focus on topics I have not yet mastered.

Given the student has completed at least one assessment in the current course,

When the adaptive engine evaluates the student's mastery score against the module's prerequisite threshold,

Then the system recommends the next unmastered module on the student's dashboard, AND deprioritizes modules already mastered above the configured threshold.

Given a student has not attempted any assessment in a course,

When the student opens their dashboard for the first time,

Then the system defaults to the course's standard entry module until sufficient mastery data is available.

User Story: As an Instructor, I want to see an automatically updated risk score for each student every week, so that I can identify and support students before they fall behind.

Given a student's engagement and assessment data for the week has been ingested into the analytics pipeline,

When the predictive model recalculates the student's risk score at the scheduled weekly batch run,

Then the updated risk score and contributing factors are displayed on the instructor's dashboard, AND students crossing the 'high risk' threshold are flagged with a red indicator.

Given a student's risk score crosses from 'medium risk' to 'high risk',

When the weekly batch job completes,

Then the system automatically sends a notification to the assigned instructor and academic advisor within 5 minutes of the score update.

User Story: As an Instructor, I want short-answer responses to be auto-graded using NLP, so that I save significant grading time on large classes.

Given an instructor has configured a model answer and grading rubric for a short-answer question,

When a student submits their response,

Then the NLP grading service returns a suggested score and highlights matched/missing key concepts within 10 seconds.

Given the NLP grading service returns a suggested score for a submission,

When the instructor disagrees and manually adjusts the score,

Then the system saves the instructor's correction and feeds it back into the model's training data to improve future grading accuracy.

4. Prioritize the Product Backlog

The Product Backlog is prioritized using a combination of Business Value (Learning Outcomes & Student Retention) and Technical Dependencies. The MoSCoW method (Must have, Should have, Could have, Won't have) is applied to ensure foundational data infrastructure is built first.

Backlog Item (User Story / Feature)	MoSCoW Priority	Business Value	Dependency / Justification
Student/instructor authentication and role-based access	MUST HAVE	High - Secures all role-based actions and student data.	Foundation for all other features. Zero dependencies.
Ingest learning activity data (LMS/clickstream events) into pipeline	MUST HAVE	High - Core data source for adaptive engine and analytics.	Depends on LMS integration. Essential for personalization.
Adaptive learning path recommendation engine (rules-based v1)	MUST HAVE	High - Directly drives personalized learning outcomes.	Depends on activity data ingestion and mastery scoring.
Early-warning risk scoring model	SHOULD HAVE	High - Enables proactive student intervention.	Depends on activity and assessment data being stable.
Automated end-of-term progress report generation	SHOULD HAVE	Medium - Administrative efficiency for instructors.	Depends on grading and analytics modules being stable.
NLP-based auto-grading for short-answer responses	COULD HAVE	Medium - Reduces grading workload, complex to build.	Depends on assessment module. Deferred to later sprints.
Predictive dropout forecasting (advanced ML)	COULD HAVE	Medium - Long-term strategic planning value.	Requires months of historical data collection first.
Parent-facing mobile app for progress digests	WON'T HAVE (Sprint 1-5)	Low - Nice to have.	Out of scope for the core learning-analytics MVP.

5. Plan the Sprint Backlog

For Sprint 1 (Duration: 2 Weeks), the team is selecting the highest-priority 'Must Have' foundational items. The goal is to establish the secure backend, database schemas, and basic ingestion of learning activity data.

Story 1: Student/instructor authentication and role-based access

- Task 1.1: Design and provision PostgreSQL schema for User/Role Management (Backend).
- Task 1.2: Integrate OAuth2/SSO for institutional login (Integration).
- Task 1.3: Develop secure REST API endpoints for authentication (Auth & Tokenization).
- Task 1.4: Develop frontend login UI for the Student and Instructor Dashboards (Frontend).

Story 2: Ingest learning activity data from LMS/clickstream events

- Task 2.1: Provision a time-series/event store (e.g., TimescaleDB) for activity data (DevOps/DBA).
- Task 2.2: Set up Apache Kafka message broker to handle high-velocity event streams (Backend).
- Task 2.3: Write a Python/Node microservice to parse LMS webhook events into Kafka (Backend).
- Task 2.4: Perform load testing simulating 5,000 concurrent student sessions (QA).

Story 3: Basic Adaptive Dashboard showing recommended next module

- Task 3.1: Develop React.js component to render the student's course progress view (Frontend).
- Task 3.2: Create a WebSocket/API connection to fetch the latest recommendation from the backend (Full-stack).
- Task 3.3: Implement basic mastery-level indicators per module on the dashboard (Frontend).

6. Estimate Effort Using Story Points

Effort estimation is performed using the Modified Fibonacci Sequence (1, 2, 3, 5, 8, 13, 21) via Planning Poker. Points represent a combination of complexity, amount of work, and risk/uncertainty. A baseline of '2 points' is equivalent to a simple, well-understood API endpoint creation.

User Story / Epic Component	Story Points	Justification & Complexity Analysis
Authentication system & database schema	8	High complexity. Requires integrating institutional SSO with our backend. Security risks are high, requiring extensive access-control testing.
Activity data ingestion via Kafka and time-series DB setup	13	Very high complexity and risk. Building a data pipeline that handles high-velocity clickstream events without bottlenecking is the most critical technical challenge of the platform.
Render adaptive dashboard and plot module recommendations	5	Medium complexity. Frontend rendering logic is well understood, but managing state updates for live mastery data requires careful React optimization.
Implement basic recommendation-fetch logic via API	3	Low-Medium complexity. Essentially a GET request that returns the next module based on a precomputed mastery flag. Logic is straightforward.
Configure CI/CD Pipelines and Cloud Environments	5	Medium complexity. Standard DevOps work, but strict data-privacy compliance (FERPA/GDPR) adds regulatory configuration overhead.

Total Sprint 1 Velocity Commitment: 34 Story Points.

7. Present the Sprint Goal and Expected Deliverables

7.1 Sprint 1 Goal

Sprint Goal: Establish the foundational system architecture capable of securely authenticating students and instructors, and successfully ingesting, processing, and displaying real-time learning activity data on an adaptive student dashboard without latency.

7.2 Expected Deliverables (End of Sprint)

At the conclusion of the 2-week sprint, the Scrum Team will present the following working deliverables during the Sprint Review:

- Secure Authentication Gateway: A working login screen where a student or instructor can authenticate via institutional SSO, backed by a secure, encrypted PostgreSQL database.
- Real-Time Learning Activity Pipeline (Backend): A fully functional data ingestion pipeline utilizing Apache Kafka and a time-series database, proven via automated load tests to handle 5,000 concurrent student sessions.
- Adaptive Dashboard Prototype (Frontend): A React-based web dashboard displaying a student's course progress. During the demo, simulated activity data flowing through the backend pipeline will update the recommended next module in real time.
- CI/CD Infrastructure: An automated deployment pipeline ensuring that code pushed to the main branch is automatically tested and deployed to the staging environment.

7.3 Definition of Done (DoD) for Sprint 1

A user story is only considered 'Done' when:

- Code is peer-reviewed and merged to the main branch.
- Unit tests are written and pass with at least 80% coverage.
- Code is deployed to the staging environment and integrated.
- The feature passes all Acceptance Criteria established in Section 3.
- Security scans (SAST/DAST) show zero critical vulnerabilities.